EX No:07

Date:

Validation Set Approach using Default Dataset

7.In Chapter 4, we used logistic regression to predict the probability of default using income and balance on the Default data set. We will now estimate the test error of this logistic regression model using the validation set approach. Do not forget to set a random seed before beginning your analysis.

LOAD LIBRARY:

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from ISLP import load_data
from ISLP.models import (ModelSpec as MS)
```

LOAD DATA:

```
Default = load_data('Default')
```

(a) Fit a logistic regression model that uses income and balance to predict default.

CODE:

```
Default['default'] = Default['default'].map({'Yes': 1, 'No': 0})
Default['student'] = Default['student'].map({'Yes': 1, 'No': 0})
x = sm.add_constant(Default[['income', 'balance']])
y = Default['default']
glm_model = sm.GLM(y, x, family=sm.families.Binomial()).fit()
print(glm_model.summary())
```

OUTPUT:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                default   No. Observations:                10000
Model:                            GLM   Df Residuals:                    9997
Model Family:                Binomial   Df Model:                           2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -789.48
Date:                Tue, 06 Oct 2026   Deviance:                       1579.0
Time:                        12:09:32   Pearson chi2:                 6.95e+03
No. Iterations:                     9   Pseudo R-squ. (CS):             0.1256
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const        -11.5405      0.435    -26.544      0.000     -12.393     -10.688
income      2.081e-05   4.99e-06      4.174      0.000     1.1e-05    3.06e-05
balance        0.0056      0.000     24.835      0.000       0.005       0.006
==============================================================================
```

(b) Using the validation set approach, estimate the test error of this model. In order to do this, you must perform the following steps:

i. Split the sample set into a training set and a validation set.

CODE:

```
testrain, validation = train_test_split( default, test_size=0.3, random_state=1 )

print('Training observations:', len(train))
print('Validation observations:', len(validation))
```

OUTPUT:

```
Training observations: 7000
Validation observations: 3000
```

ii. Fit a multiple logistic regression model using only the training observations.

CODE:

```
x_train = sm.add_constant(train[['income', 'balance']])
y_train = train['default']
model = sm.GLM(y_train, x_train, family=sm.families.Binomial()).fit()
print(model.summary())
```

OUTPUT:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                 default   No. Observations:                 7000
Model:                             GLM   Df Residuals:                     6997
Model Family:                Binomial   Df Model:                            2
Link Function:                   Logit   Scale:                          1.0000
Method:                           IRLS   Log-Likelihood:                -547.79
Date:                 Thu, 15 Oct 2026   Deviance:                       1095.6
Time:                         18:45:32   Pearson chi2:                 5.92e+03
No. Iterations:                      9   Pseudo R-squ. (CS):             0.1317
Covariance Type:             nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const        -11.6019      0.522    -22.236      0.000     -12.624     -10.579
income      1.872e-05   5.92e-06      3.163      0.002    7.12e-06    3.03e-05
balance        0.0057      0.000     20.976      0.000       0.005       0.006
==============================================================================
```

iii. Obtain a prediction of default status for each individual in the validation set by computing the posterior probability of default for that individual, and classifying the individual to the default category if the posterior probability is greater than 0.5.

CODE:

```
X_test = test[['income', 'balance']]
X_test = sm.add_constant(X_test)
pred = model.predict(X_test)
pred=(pred > 0.5).astype(int)
```

iv. Compute the validation set error, which is the fraction of the observations in the validation set that are misclassified.

CODE:

```
y_test=(test['default']==1).astype(int)
print("Test set error rate: ", np.mean(pred != y  test))
accuracy = np.mean(pred == y_test)
print("Test set accuracy: ", accuracy)
```

OUTPUT:

```
Test set error rate:  0.02666666666666667
Test set accuracy:  0.9733333333333334
```

(c) Repeat the process in (b) three times, using three different splits of the observations into a training set and a validation set. Comment on the results obtained.

CODE:

```
test_errors = []
for seed in [1, 2, 3]:
    X_train, X_test, y_train, y_test = train_test_split( x, y, test_size=0.3, random_state=seed  )
    X_train = x.loc[X_train.index]
    X_test = x.loc[X_test.index]
    glm_results = sm.GLM(y_train, X_train).fit()
    probs = glm_results.predict(X_test)
    preds = (probs > 0.5).astype(int)
    test_error = np.mean(preds != y_test)
    test_errors.append(test_error)
    print(f"Test error (seed={seed}): {test_error:.4f}")

print("Average test error:", np.mean(test_errors))
```

OUTPUT:

```
Test error (seed=1): 0.0303
Test error (seed=2): 0.0253
Test error (seed=3): 0.0353
Average test error: 0.030333333333333334
```

(d) Now consider a logistic regression model that predicts the probability of default using income, balance, and a dummy variable for student. Estimate the test error for this model using the validation set approach. Comment on whether or not including a dummy variable for student leads to a reduction in the test error rate

CODE:

```
X2 = Default[['income', 'balance', 'student']]
test_errors_student = []

for seed in [1, 2, 3]:
    X_train, X_test, y_train, y_test = train_test_split(
        X2, y, test_size=0.3, random_state=seed
    )
```

```
    X_train_I = MS(X_train).fit_transform(X_train)
    X_test_I = MS(X_test).fit_transform(X_test)

    glm_model_val = sm.GLM(y_train, X_train_I, family=sm.families.Binomial()).fit()
    glm_results_val = glm_model_val.summary()

    probs = glm_model_val.predict(X_test_I)
    preds = (np.where(probs > 0.5, 1, 0))
    val_error = np.mean(preds != y_test)
    test_errors_student.append(val_error)
    print(f"Test error with student (seed={seed}): {val_error:.4f}")

print("Average test error with student variable:", np.mean(test_errors_student))
```

OUTPUT:

```
Test error with student (seed=1): 0.0243
Test error with student (seed=2): 0.0237
Test error with student (seed=3): 0.0247
Average test error with student variable: 0.024222222222222225
```

INTERPRETATION:

- (a) In the logistic regression on all 10,000 customers, both income (2.08e-05, $p < 0.001$) and balance (0.0056, $p < 0.001$) are significant, but balance is far stronger ($z = 24.8$ against 4.2). Each extra dollar of balance raises the log-odds of default by 0.0056, so an extra $100 multiplies the odds of default by about 1.75.
- (b) With a 70/30 split (7,000 training and 3,000 validation observations), the validation-set error is 2.67% (accuracy 97.33%). The coefficients on the training set (income 1.87e-05, balance 0.0057) are close to those from the full data.
- (c) Repeating the split with seeds 1, 2 and 3 gives validation errors of 3.03%, 2.53% and 3.53% (average 3.03%). The estimate moves by about one percentage point depending on the split, so a single validation set gives a variable estimate, and it trains on only 70% of the data, which tends to overstate the error.
- (d) Adding the student dummy gives validation errors of 2.43%, 2.37% and 2.47% (average 2.42%), lower than 3.03% without it, which suggests a small gain. However, the loop in (c) calls sm.GLM without family=Binomial (a linear-probability fit) while (d) uses the Binomial family, so part of the gap may come from the model form rather than from student; the two should be compared using the same model.
- Only about 3.3% of customers default, so always predicting 'No default' would already give an error of about 3.3%; the model's 2.4 to 3.5% error is only modestly better than that baseline.